

LeetCode編碼：

706

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔





- 不使用內建hash map，實作一個支援以下三種操作的 HashMap：
`put(key, value)` 插入或更新鍵值對、
`get(key)` 查詢鍵對應的值（不存在時回傳 -1）、
`remove(key)` 刪除指定鍵。題目保證所有 `key` 的範圍在 0 到 10^6 之間，`value` 為非負整數。
- 以 `put(1, 1)`、`put(2, 2)`、`get(1)`、`get(3)`、`put(2, 1)`、`get(2)`、`remove(2)`、`get(2)` 為例。
 前兩次 `put` 分別插入鍵 1 和鍵 2。
 第一次 `get(1)` 回傳 1，`get(3)` 回傳 -1，因為鍵 3 從未被插入。
`put(2, 1)` 將鍵 2 的值從 2 更新為 1，更新非重複插入，若沒有先搜尋再覆蓋，就會出現同一個 `key` 存在兩筆資料的錯誤。
`get(2)` 此時回傳更新後的 1。`remove(2)` 刪除鍵 2 後，最後的 `get(2)` 回傳 -1。
- 需注意 `put` 時若 `key` 已存在，必須先找到舊值覆蓋並非直接新增，否則同一個 `key` 會有多個 `value` 共存，導致 `get` 和 `remove` 行為錯誤。



- 解法一：直接定址陣列
- 利用題目已知 **key** 的範圍在 0 到 10^6 之間這個條件，直接設一個大小為 $10^6 + 1$ 的陣列，讓 **key** 直接作為陣列的索引來存取對應的 **value**。用 -1 表示該位置尚未被插入。

好處是所有操作（**put**、**get**、**remove**）都是直接陣列存取，時間複雜度嚴格為 **$O(1)$** ，且不需要碰撞處理，實作簡單。

因為 **key** 的範圍有明確上界，且上界不算太大（約 4MB），在記憶體允許的情況下是合理的取捨。缺點是無論實際插入了幾筆資料，都固定消耗 **$O(N_{\max})$** 的空間，若實際使用的 **key** 只有幾個，大部分空間都會浪費。

典型的以空間換時間，這個設計在 **key** 範圍不確定或範圍極大的情境下就不適用。



- 解法二：Separate Chaining
- 在不預先知道 key 範圍的情況下，用有限的空間支援雜湊表的所有操作，同時展示雜湊表如何處理 Collision。
- 先設固定大小的 bucket 陣列，每個 bucket 裡存放一個 list，list 中放的是所有 hash 到這個 bucket 的 [key, value]。
新增時，先用 $\text{key} \% \text{bucket_size}$ 計算出對應的 bucket，再線性搜尋 bucket 中是否已有相同的 key，若有則覆蓋 value，若無則 append 新的 pair。查詢與刪除理。
- bucket 大小用質數是為了讓不同的 key 盡量均勻分散到不同的 bucket，減少碰撞的機率。

平均情況下每個 bucket 只有少數幾個元素，線性搜尋接近 $O(1)$ ；最差情況下所有 key 都碰撞到同一個 bucket，退化為 $O(n)$ 。

空間複雜度為 $O(n + \text{buckets})$ ，其中 n 是插入的 key 數量，比直接定址法節省空間。



- 直接定址法展示了雜湊表最理想的情況：**key** 可以直接對應到記憶體位置，**O(1)** 存取不需要任何額外處理；
Separate Chaining 則展示了 **key** 範圍未知或過大時的標準解以及碰撞是雜湊表設計的難題。

方法	Time	Space
直接定址	O(1)	O(10^6)
Chaining	O(1) avg	O(n)



```

1 class MyHashMap(object):
2     import heapq # 此題不需要，僅示意位置一致
3
4     def __init__(self):
5         # 解法一：直接定址陣列
6         # key 範圍為 0~10^6，開這個大小確保所有合法 key 都有對應位置
7         # 用 -1 表示該位置尚未被插入，因為 value 保證為非負整數
8         self.data = [-1] * (10**6 + 1)
9
10        # 解法二：Separate Chaining
11        # 選質數 1009 是為了讓 key 均勻分散到各個 bucket
12        # 若用合數，規律性的 key 容易集中碰撞到同一個 bucket
13        self.size = 1009
14        self.buckets = [[] for _ in range(self.size)]
15
16    def _hash(self, key):
17        return key % self.size
18
19    # 解法一：直接定址
20    def _put_direct(self, key, value):
21        self.data[key] = value
22
23    def _get_direct(self, key):
24        return self.data[key]
25
26    def _remove_direct(self, key):
27        self.data[key] = -1
28

```

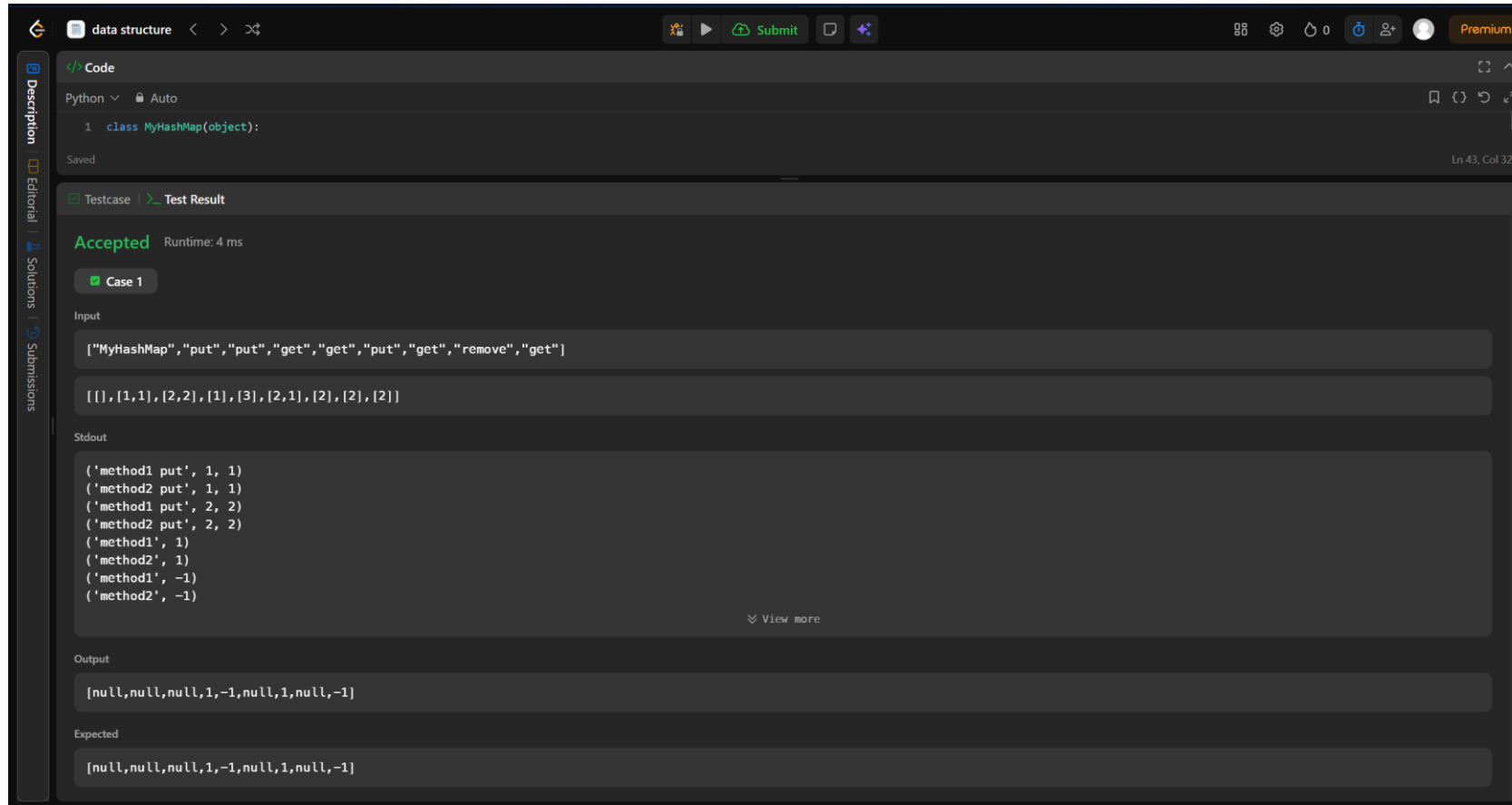
```

26 def _remove_direct(self, key):
27     self.data[key] = -1
28
29 # 解法二：Separate Chaining
30 def _put_chaining(self, key, value):
31     bucket = self.buckets[self._hash(key)]
32     for pair in bucket:
33         if pair[0] == key:
34             # key 已存在則覆蓋 value，避免同一個 key 有多筆資料
35             pair[1] = value
36             return
37     bucket.append([key, value])
38
39 def _get_chaining(self, key):
40     for pair in self.buckets[self._hash(key)]:
41         if pair[0] == key:
42             return pair[1]
43     # 找不到 key 回傳 -1，與題目定義一致
44     return -1
45
46 def _remove_chaining(self, key):
47     bucket = self.buckets[self._hash(key)]
48     for i, pair in enumerate(bucket):
49         if pair[0] == key:
50             bucket.pop(i)
51             return
52
53 def put(self, key, value):
54     self._put_direct(key, value)
55     self._put_chaining(key, value)
56     print("method1 put", key, value)
57     print("method2 put", key, value)
58
59 def get(self, key):
60     res1 = self._get_direct(key)
61     res2 = self._get_chaining(key)
62
63     print("method1", res1)
64     print("method2", res2)
65
66     return res1
67
68 def remove(self, key):
69     self._remove_direct(key)
70     self._remove_chaining(key)
71     print("method1 remove", key)
72     print("method2 remove", key)

```



- 結果證明：



data structure < > > Submit

Code

Python Auto

```
1 class MyHashMap(object):
```

Saved Ln 43, Col 32

Testcase Test Result

Accepted Runtime: 4 ms

Case 1

Input

```
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
```

```
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
```

Stdout

```
('method1 put', 1, 1)
('method2 put', 1, 1)
('method1 put', 2, 2)
('method2 put', 2, 2)
('method1', 1)
('method2', 1)
('method1', -1)
('method2', -1)
```

View more

Output

```
[null, null, null, 1, -1, null, 1, null, -1]
```

Expected

```
[null, null, null, 1, -1, null, 1, null, -1]
```



LeetCode編碼：

146

系所： 北聯(北醫) 生物醫學工程學系
學號： B812111001
姓名： 張博翔





設計一個容量為 `capacity` 的 LRU (Least Recently Used) cache。

支援兩種操作：`get(key)` 查詢鍵對應的值，若不存在回傳 `-1`；`put(key, value)` 插入或更新鍵值對。

兩種操作都必須在 $O(1)$ 時間內完成。當快取已滿且需要插入新鍵時，清除最久未被使用的資料再插入新的。

LRU 是最近用過的留下，最久沒用的優先清除，也是實際系統中快取的行為：最近存取過的資料很可能很快又被存取，所以優先保留；長期沒被碰過的資料可能已經不需要了。

- 以 `capacity = 2` 為例，依序執行 `put(1,1)`、`put(2,2)`、`get(1)`、`put(3,3)`、`get(2)`。
`put(1,1)` 與 `put(2,2)` 分別插入兩筆，快取已滿，使用順序為 `[1, 2]`。
`get(1)` 存取 `key 1`，`key 1` 變成最近使用，順序更新為 `[2, 1]`。
`put(3,3)` 要插入新鍵但快取已滿，此時最久未使用的是 `key 2`，清除 `key 2` 後插入 `key 3`，順序為 `[1, 3]`。
 最後 `get(2)` 查詢 `key 2`，因為已被清除因此回傳 `-1`。
- `put` 時若 `key` 已存在，不只需要更新 `value` 還必須將這個 `key` 移到最近使用，因為更新本身也算一次存取。
 刪除 LRU 節點時，`HashMap` 也必須同步刪除，否則後續 `get` 仍能查到已被清除的資料。



- 解法一： **OrderedDict**
- Python 的 `collections.OrderedDict` 是一個記錄插入順序的有序字典，提供 `move_to_end(key)` 將指定鍵移到最尾端，以及 `popitem(last=False)` 從最前端移除最舊的鍵，這兩個操作都是 $O(1)$ ，非常適合 LRU 需求。
- 每次 `get` 或 `put` 時，只要呼叫 `move_to_end(key)` 將該鍵標記為「最近使用」，維持尾端是最新、頭端是最舊的不變式。當 `put` 導致超過容量時，呼叫 `popitem(last=False)` 從頭端移除最久未使用的鍵。

這個方法的優點是程式碼短，三行核心邏輯就能完成。美中不足就是 **OrderedDict** 是已封裝好的資料結構，少了設計結構的過程。



- 解法二： **HashMap + Doubly Linked List (DLL)**
- HashMap 與 DLL 這兩種結構分別解決了 LRU 的兩個核心。
HashMap 負責 $O(1)$ 的鍵值查找，key 對應到 DLL 中的節點指標，讓我們能在常數時間內找到任意節點。
DLL 負責維護使用順序，因為DLL的節點可以在 $O(1)$ 內完成插入與刪除，而Singly Linked List 刪除節點需要從頭找到前一個節點，需要 $O(n)$ 。
- DLL 尾端代表最近使用，頭端代表最久未使用。每次存取或更新一個 key 時，先透過 HashMap 找到節點從 DLL 中移除，再插到尾端代表它是最近被使用的。
當快取滿了需要清除時，直接移除 DLL 頭端的節點，並從 HashMap 中刪除對應 key。
使用 **dummy head** 與 **dummy tail** 是為了消除邊界判斷。若沒有虛擬節點，插入與刪除時需要特別處理當前節點是否為頭部或尾部的情况，程式碼複雜。



- 同時需要 $O(1)$ 查找與 $O(1)$ 順序維護，而沒有任何單一資料結構能同時滿足這兩個需求。
 HashMap 單獨使用時無法維護順序；DLL 單獨使用時查找需要 $O(n)$ 。
 將兩者結合，讓 HashMap 存指標、DLL 存順序，各自負責擅長的部分，才能同時達到兩個 $O(1)$ 。

方法	Time	Space
OrderedDict	$O(1)$	$O(\text{cap})$
HashMap+DLL	$O(1)$	$O(\text{cap})$



- 合併太複雜，分開寫：

```

1 from collections import OrderedDict
2
3 class LRUCache(object):
4
5     def __init__(self, capacity):
6         """
7         :type capacity: int
8         """
9         self.cap = capacity
10        self.cache = OrderedDict()
11
12    def get(self, key):
13        """
14        :type key: int
15        :rtype: int
16        """
17        if key not in self.cache:
18            return -1
19        # 存取過就移到尾端，維持尾端最新、頭端最舊的不變式
20        value = self.cache.pop(key)
21        self.cache[key] = value
22        return value
23
24    def put(self, key, value):
25        """
26        :type key: int
27        :type value: int
28        :rtype: None
29        """
30        if key in self.cache:
31            # 更新已存在的 key 也算一次存取，需移到尾端
32            self.cache.pop(key)
33        self.cache[key] = value
34        if len(self.cache) > self.cap:
35            # 頭端是最久未使用的，last=False 確保從頭端移除
36            self.cache.pop(next(iter(self.cache)))
37

```

```

1 # 解法二：HashMap + 雙向鏈結串列
2 class Node:
3     def __init__(self, k=0, v=0):
4         self.key = k; self.val = v
5         self.prev = self.next = None
6
7 class LRUCache(object):
8
9     def __init__(self, capacity):
10        """
11        :type capacity: int
12        """
13        self.cap = capacity
14        self.map = {}
15        # 虛擬頭尾節點讓所有真實節點都有前後節點
16        # 避免插入刪除時需要特判頭部或尾部的邊界情況
17        self.head = Node(); self.tail = Node()
18        self.head.next = self.tail
19        self.tail.prev = self.head
20
21    def _remove(self, node):
22        # 直接修改前後節點的指標，不需要從頭搜尋
23        # 這是選用雙向而非單向鏈結串列的原因
24        node.prev.next = node.next
25        node.next.prev = node.prev
26
27    def _add_tail(self, node):
28        # 插入到虛擬尾節點之前，代表這個節點是最近使用的
29        node.prev = self.tail.prev
30        node.next = self.tail
31        self.tail.prev.next = node
32        self.tail.prev = node
33

```

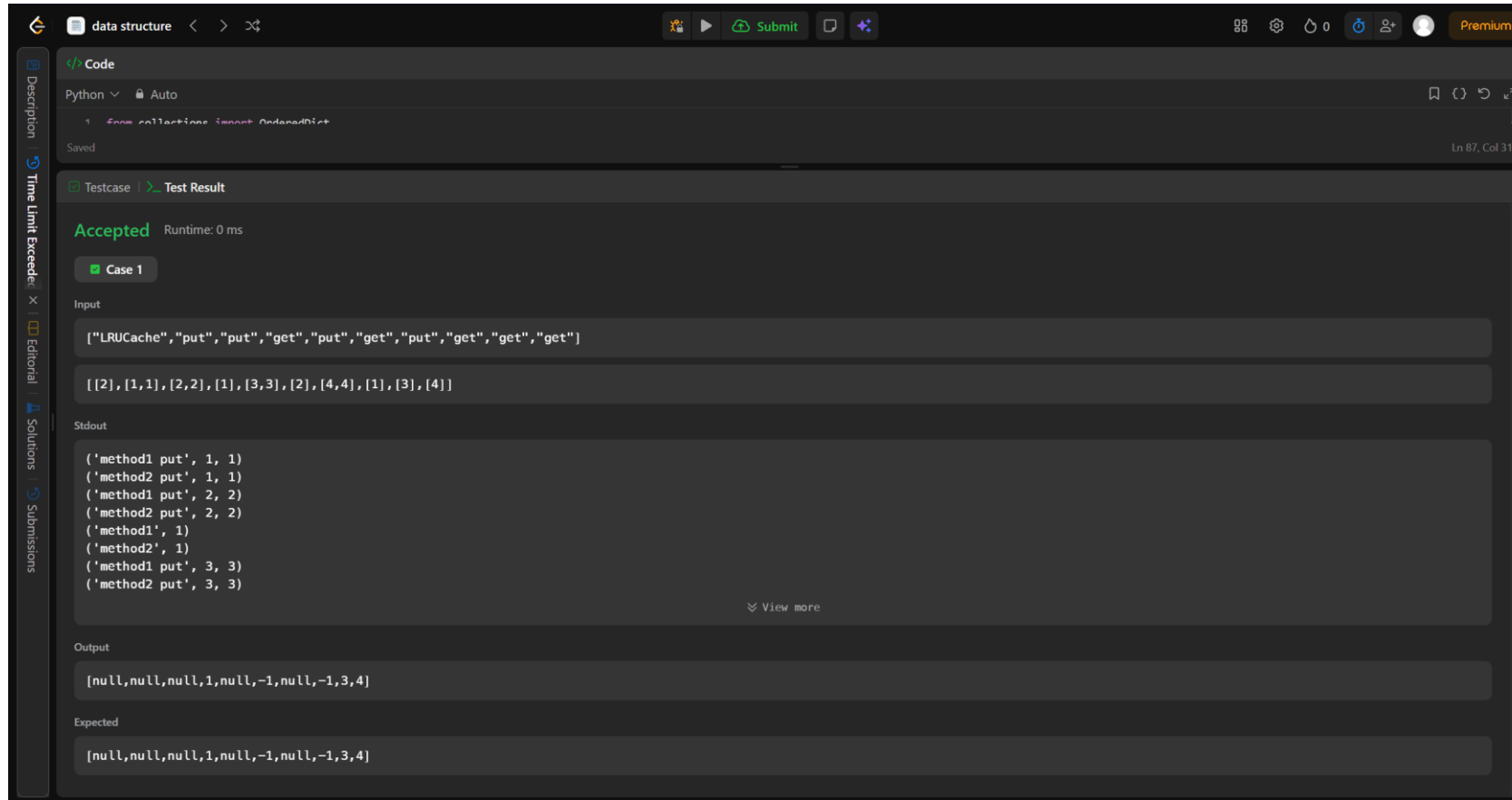
```

34    def get(self, key):
35        if key not in self.map:
36            return -1
37        n = self.map[key]
38        # 存取後移到尾端，表示最近使用
39        self._remove(n); self._add_tail(n)
40        return n.val
41
42    def put(self, key, value):
43        if key in self.map:
44            n = self.map[key]; n.val = value
45            self._remove(n); self._add_tail(n)
46        else:
47            if len(self.map) == self.cap:
48                # 頭端的下一個是最久未使用的真實節點
49                lru = self.head.next
50                self._remove(lru)
51                # HashMap 也需同步刪除，否則後續 get 仍能查到已淘汰的資料
52                del self.map[lru.key]
53            n = Node(key, value)
54            self.map[key] = n
55            self._add_tail(n)
56

```



- 結果證明：



data structure < > ⚙

Submit

Code

Python Auto

Ln 87, Col 31

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

```
["LRUCache","put","put","get","put","get","put","get","get","get"]
[[2],[1,1],[2,2],[1],[3,3],[2],[4,4],[1],[3],[4]]
```

Stdout

```
('method1 put', 1, 1)
('method2 put', 1, 1)
('method1 put', 2, 2)
('method2 put', 2, 2)
('method1', 1)
('method2', 1)
('method1 put', 3, 3)
('method2 put', 3, 3)
```

View more

Output

```
[null,null,null,1,null,-1,null,-1,3,4]
```

Expected

```
[null,null,null,1,null,-1,null,-1,3,4]
```



LeetCode編碼：

1941

系所： 北聯(北醫) 生物醫學工程學系
學號： B812111001
姓名： 張博翔



題目 (含解釋&說明)



給定只包含小寫英文字母的非空字串 s ，判斷字串中所有出現過的字元，其出現次數是否全部相同。

- 以 $s = \text{"abacbc"}$ 為例：a 出現 2 次、b 出現 2 次、c 出現 2 次，三種字元次數相同，回傳 True。
- 以 $s = \text{"aaabb"}$ 為例：a 出現 3 次、b 出現 2 次，次數不同回傳 False。



- 解法一：Counter + set
- 先統計每個字元的出現次數，再判斷所有次數是否相同。使用 `collections.Counter` 統計各字元頻率後，將所有頻率轉成 `set`。
`set` 會去除重複值，因此若所有字元的出現次數都一樣，`set` 的大小就會是 1；若有任何兩個字元的次數不同，`set` 的大小就會大於 1。

`Counter` 與 `set` 都是 `Python` 的內建工具，兩者搭配能用少量程式碼表達。
字元集最多 26 個，所有操作的實際規模都是常數，整體時間複雜度為 $O(n)$ （ n 為字串長度），空間複雜度為 $O(k)$ （ k 為不同字元種類數，最多 26）。



- 解法二：**Frequency of Frequency**
- 在解法一的基礎上多加一層統計，進一步計算有幾種不同的出現次數，其實就是對頻率再做一次 **Counter**。
- 用 **Counter** 得到每個字元的頻率，再對這些頻率值再做一次 **Counter**，得到每種頻率出現了幾次。
若所有字元的次數都相同，第二層 **Counter** 就只會有一個鍵，大小為 1，回傳 **True**；若有不同回傳 **False**。
- 與解法一的結果等價（ `len(set(values)) == 1` 同理於 `len(Counter(values)) == 1` ），但價值在於展示更通用技巧。

二層統計結構在更複雜的題目中能提供更多資訊，例如 **LC 347** 或 **LC 451**，當單靠一層 **Counter** 加 **set** 不夠用時二層結構的優勢就會顯現。



方法	Time	Space
Counter+set	$O(n)$	$O(k)$
Freq of Freq	$O(n)$	$O(k)$



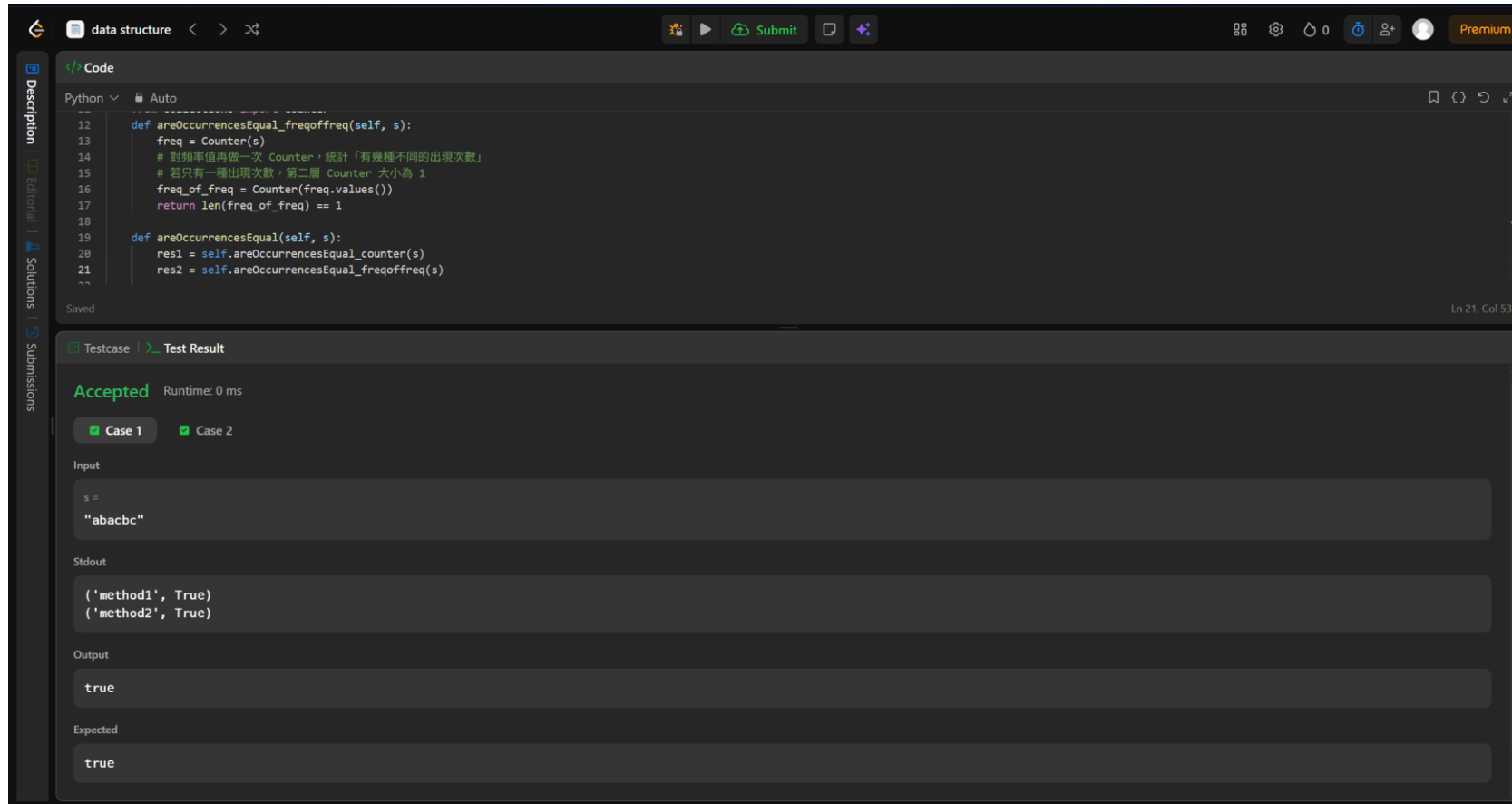
```

1 class Solution(object):
2     # 解法一：Counter + set
3     from collections import Counter
4     def areOccurrencesEqual_counter(self, s):
5         freq = Counter(s)
6         # set 會去除重複值，若所有頻率相同則 set 大小為 1
7         # 大於 1 代表存在至少兩種不同的出現次數
8         return len(set(freq.values())) == 1
9
10    # 解法二：頻率的頻率
11    from collections import Counter
12    def areOccurrencesEqual_freqoffreq(self, s):
13        freq = Counter(s)
14        # 對頻率值再做一次 Counter，統計「有幾種不同的出現次數」
15        # 若只有一種出現次數，第二層 Counter 大小為 1
16        freq_of_freq = Counter(freq.values())
17        return len(freq_of_freq) == 1
18
19    def areOccurrencesEqual(self, s):
20        res1 = self.areOccurrencesEqual_counter(s)
21        res2 = self.areOccurrencesEqual_freqoffreq(s)
22
23        print("method1", res1)
24        print("method2", res2)
25
26        return res1

```



- 結果證明：



The screenshot shows a code editor interface with a dark theme. The top bar includes a file explorer, a search bar, and a 'Submit' button. The main editor area displays Python code for a class method. The code defines two methods: `areOccurrencesEqual_freqoffreq` and `areOccurrencesEqual`. The first method uses `Counter` to count the frequency of each character in a string `s` and then counts the frequency of those counts. The second method uses `areOccurrencesEqual_counter` to count the frequency of each character in `s` and then checks if the length of the frequency list is 1. The test results section shows that the code is 'Accepted' with a runtime of 0 ms. The input is `s = "abacbc"`, the stdout is `('method1', True)` and `('method2', True)`, the output is `true`, and the expected output is `true`.

```

Python 3.10.0
def areOccurrencesEqual_freqoffreq(self, s):
    freq = Counter(s)
    # 對頻率值再做一次 Counter，統計「有幾種不同的出現次數」
    # 若只有一種出現次數，第二層 Counter 大小為 1
    freq_of_freq = Counter(freq.values())
    return len(freq_of_freq) == 1

def areOccurrencesEqual(self, s):
    res1 = self.areOccurrencesEqual_counter(s)
    res2 = self.areOccurrencesEqual_freqoffreq(s)
    return res1 == res2

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

s =
"abacbc"

Stdout

('method1', True)
('method2', True)

Output

true

Expected

true